Math 118: Mathematical Methods of Data Theory

Lecture 19: Dynamic Programming 2

Instructor: Daniel Mckenzie

Dept. of Mathematics, UCLA

February 26, 2020

Dynamic Programming thus far

- An algorithmic framework for **discrete optimization**.
- Works when **objective function** f is **recursively definable** in terms of smaller sub-problems.
- When there are **overlapping sub-problems**, dynamic programming is much faster than brute-force search.

Example: Fibonacci Numbers

- **Recall:** n-th Fibonacci number defined as $f(n) = f(n-1) + f(n-2)$.
- Consider the following brute-force, top-down approach to computing $f(n)$:
- Complexity $= O(2^n)$
- **Note:** there are overlapping subproblems.

Example: Fibonacci Numbers

- Dynamic Programming approach: Use $f(n) = f(n-1) + f(n-2)$ and fill table from bottom-up:
-

$f(1)$	$f(2)$	$f(3)$	$f(4)$	$f(5)$	$\ldots$
0	1				

-

$f(1)$	$f(2)$	$f(3)$	$f(4)$	$f(5)$	$\ldots$
0	1	1			

-

$f(1)$	$f(2)$	$f(3)$	$f(4)$	$f(5)$	$\ldots$
0	1	1	2		

-

$f(1)$	$f(2)$	$f(3)$	$f(4)$	$f(5)$	$\ldots$
0	1	1	2	3	$\ldots$

- Complexity is $O(n)$.

Example: The Knapsack Problem

- You are going hiking and have a bag (*i.e.* knapsack) of fixed volume. Need to fill it with items of various sizes and values.
- **More Formally:**
  - Volume of knapsack $= B \in \mathbb{Z}_{>0}$.
  - n items of sizes $s_1, \ldots, s_n \in \mathbb{Z}_{>0}$ and values $v_1 \ldots, v_n \in \mathbb{Z}_{>0}$.
  - **Goal:** Find subset $A \subset \{1, \ldots, n\}$ maximizing $V(A) = \sum_{i \in A} v_i$ subject to: $\displaystyle\sum_{i \in A} s_i \leq B$.
- **Questions:**
  - What is the objective function?
  - **Answer:** $\displaystyle f(n, B) = \max_{A \subset \{1,\ldots,n\}} \left\{ V(A) \text{ subject to: } \sum_{i \in A} s_i \leq B \right\}$.
  - What are the sub-problems?

Example: The Knapsack Problem

- **(Sub-)Problem:** Compute $f(k, R) = \max_{A \subset \{1,\ldots,k\}} \left\{ V(A) : \sum_{i \in A} s_i \leq R \right\}$
- **Ultimate Goal:** compute $f(n, B)$.
- Consider last item n. Two choices:
  - **If at least s_n space remaining**: add to bag. Reduce available space to $B - s_n$. Reduce number of items by 1. Increase value by v_n.
  - **If less than s_n space remaining**: discard. Neither value nor available space change. Number of items reduced by 1.
- In both cases, reduce computing $f(n)$ to $f(n-1)$:

$$f(n, B) = \max \begin{cases} f(n-1, B - s_n) + v_n & \text{if } s_n \leq B \\ f(n-1, B) \end{cases}$$

- In general, for $R \leq B$:

$$f(k, R) = \max \begin{cases} f(k-1, R-s_k) + v_k & \text{if } s_k \leq R \\ f(k-1, R) \end{cases}$$

- **Base Case(s):** $f(0, R) = 0$ for any R.
- **Dynamic Programming Strategy:**
  - Construct $(k+1) \times (B+1)$ table.
  - Fill in $f(0, R) = 0$ for all $0 \leq R \leq B$.
  - Continue recursively. Maximum achievable value in $(k+1, B+1)$ cell.
  - Traceback to find optimal set of items, A.

Knapsack Problem: An example

Example

Consider the knapsack problem with $B = 5$ and four items:

$$s_1 = 2, v_1 = 3; \quad s_2 = 3, v_2 = 4; \quad s_3 = 4, v_3 = 5; \quad s_4 = 5, v_4 = 6$$

Item ID $\downarrow$ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

- **Recall:** $s_1 = 2, v_1 = 3; \quad s_2 = 3, v_2 = 4; \quad s_3 = 4, v_3 = 5; \quad s_4 = 5, v_4 = 6$
- $f(k, 1) = \max \begin{cases} 0 + v_k & \text{if } s_k \leq 1 \\ 0 \end{cases}$

Item ID $\downarrow$ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0				
2	0	0				
3	0	0				
4	0	0				

- **Recall:** $s_1 = 2, v_1 = 3; \quad s_2 = 3, v_2 = 4; \quad s_3 = 4, v_3 = 5; \quad s_4 = 5, v_4 = 6$
- $f(1,2) = \max \begin{cases} 0 + v_1 & \text{if } s_1 \leq 2 \\ 0 \end{cases} = 2$
- For $k \geq 2$: $f(k,2) = \max \begin{cases} 0 + v_k & \text{if } s_k \leq 2 \\ f(k-1,2) \end{cases} =$

  $\max \begin{cases} 0 + v_k & \text{if } s_k \leq 2 \\ 3 \end{cases} = 3$

Item ID $\downarrow$ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3			
2	0	0	3			
3	0	0	3			
4	0	0	3			

- **Recall:** $s_1 = 2, v_1 = 3; \quad s_2 = 3, v_2 = 4; \quad s_3 = 4, v_3 = 5; \quad s_4 = 5, v_4 = 6$
- $f(1, 3) = 3$
- $f(2, 3) = \max \begin{cases} f(2-1, 3-s_2) + v_2 & \text{if } s_2 \leq 3 \\ f(2-1, 3) \end{cases} = \max \begin{cases} 0+4 \\ 3 \end{cases} = 4$
- Can Check: for $k \geq 3$: $f(k, 3) = 4$.

Item ID $\downarrow$ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3		
2	0	0	3	4		
3	0	0	3	4		
4	0	0	3	4		

- **Recall:** $s_1 = 2, v_1 = 3; \quad s_2 = 3, v_2 = 4; \quad s_3 = 4, v_3 = 5; \quad s_4 = 5, v_4 = 6$
- $f(2,4) = \max \begin{cases} f(2-1, 4-s_2) + v_2 & \text{if } s_2 \leq 4 \\ f(2-1,4) \end{cases} = \max \begin{cases} 0+4 \\ 3 \end{cases} = 4$
- *Exercise:* Can check that $f(3,4) = f(4,4) = 5$.

Item ID ↓ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	
2	0	0	3	4	4	
3	0	0	3	4	5	
4	0	0	3	4	5	

- **Recall:** $s_1 = 2, v_1 = 3; \quad s_2 = 3, v_2 = 4; \quad s_3 = 4, v_3 = 5; \quad s_4 = 5, v_4 = 6$
- $f(2,5) = \max \begin{cases} f(2-1, 5-s_2) + v_2 & \text{if } s_2 \leq 3 \\ f(2-1,5) \end{cases} = \max \begin{cases} 3+4 \\ 3 \end{cases} = 7$
- *Exercise:* Can check that $f(3,5) = f(4,5) = 7$.

Item ID $\downarrow$ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Traceback

- **Recall:** $s_1 = 2, v_1 = 3; \quad s_2 = 3, v_2 = 4; \quad s_3 = 4, v_3 = 5; \quad s_4 = 5, v_4 = 6$

Item ID ↓ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

- **Question:** How did I get here?
- **Recall:** $f(k, R) = \max \begin{cases} f(k-1, R-s_k) + v_k & \text{if } s_k < R \\ f(k-1, R) \text{ otherwise} \end{cases}$
- $f(4, 5) = \max \begin{cases} f(3, 0) + v_4 \\ f(3, 5) \end{cases} = \max \begin{cases} 0 + 6 \\ 7 \end{cases}$

- **Recall:** $s_1 = 2, v_1 = 3; \quad s_2 = 3, v_2 = 4; \quad s_3 = 4, v_3 = 5; \quad s_4 = 5, v_4 = 6$

Item ID $\downarrow$ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

- **Question:** How did I get here?
- **Recall:** $f(k, R) = \max \begin{cases} f(k-1, R-s_k) + v_k & \text{if } s_k < R \\ f(k-1, R) \text{ otherwise} \end{cases}$
- $f(3,5) = \max \begin{cases} f(2,1) + v_3 \\ f(2,5) \end{cases} = \max \begin{cases} 0 + 5 \\ 7 \end{cases}$

- **Recall:** $s_1 = 2, v_1 = 3;$ $s_2 = 3, v_2 = 4;$ $s_3 = 4, v_3 = 5;$ $s_4 = 5, v_4 = 6$

Item ID $\downarrow$ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Continuing in this fashion, we get:

- **Recall:** $s_1 = 2, v_1 = 3; \quad s_2 = 3, v_2 = 4; \quad s_3 = 4, v_3 = 5; \quad s_4 = 5, v_4 = 6$

Item ID ↓ \ $B \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

- Making diagonal move is only time we add items to backpack.
- Therefore, items included in optimal set correspond to rows **arrived at** by a diagonal move.
- In this case, **optimal set** is $A = \{1, 2\}$.